Multilayer Perceptron (MLP)

Fundamentos Teóricos e Implementação Prática

Baseado em: *Deep Learning* — Goodfellow, Bengio & Courville (2016)

Disciplina: Aprendizado de Máquina Profundo

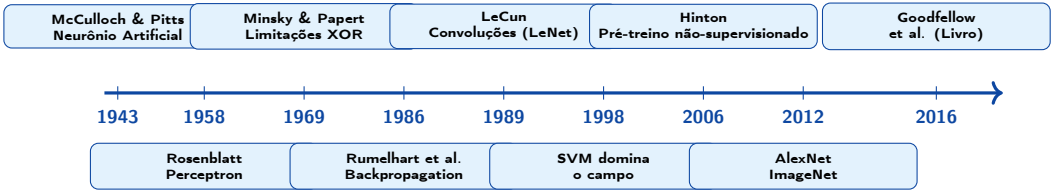
Graduação em Ciência da Computação / Engenharia

2025

Agenda da Aula

1. Introdução Histórica
2. Formalismo Matemático
3. Funções de Ativação
4. O Problema XOR e a Necessidade de Não-Linearidade
5. Design de Arquiteturas
6. Funções de Perda
7. Backpropagation e Grafos Computacionais
8. Regularização
9. Introdução ao PyTorch
10. Implementação: MNIST com PyTorch

Linha do Tempo: Das Origens às Redes Profundas

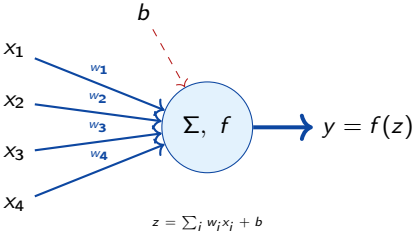


O MLP é o **bloco fundamental** de todo o *deep learning* moderno.

O Neurônio Biológico e o Modelo Matemático

Neurônio Biológico

- **Dendritos**: recebem sinais de entrada
- **Corpo celular**: integra sinais (soma ponderada)
- **Axônio**: transmite saída
- **Sinapse**: peso da conexão (plasticidade)
- Dispara apenas se limiar é atingido (*threshold*)



Modelo McCulloch-Pitts (1943) → Perceptron (Rosenblatt, 1958)

O Perceptron introduziu **pesos ajustáveis** e uma **regra de aprendizado**; o MLP estende isso a múltiplas camadas com funções de ativação diferenciáveis.

Definição Formal de um MLP

Definição (Goodfellow et al., Cap. 6)

Um MLP com L camadas define uma função composta:

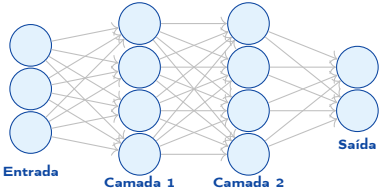
$$f(\boldsymbol{x};\boldsymbol{\theta}) = f^{(L)}\big(f^{(L-1)}(\cdots f^{(1)}(\boldsymbol{x})\cdots)\big)$$

onde cada camada ℓ computa:

$$\boxed{\boldsymbol{h}^{(\ell)} = f^{(\ell)}\big(\boldsymbol{z}^{(\ell)}\big), \qquad \boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}}$$

Notação

$\boldsymbol{x} \in \mathbb{R}^{d_0}$	vetor de entrada
$\boldsymbol{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$	matriz de pesos
$\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$	vetor de vieses
$\boldsymbol{h}^{(\ell)} \in \mathbb{R}^{d_\ell}$	ativação da camada ℓ
$f^{(\ell)}$	função de ativação
$\boldsymbol{\theta} = \{\boldsymbol{W}^{(\ell)}, \boldsymbol{b}^{(\ell)}\}_\ell$	todos os parâmetros



Cálculo Manual — Rede Pequena (Exemplo Detalhado)

Configuração: 2 entradas → 2 neurônios ocultos → 1 saída

Ativação oculta: ReLU($\cdot$); saída: sigmoide $\sigma(\cdot)$.

Parâmetros dados:

$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 1 & 2 \\ -1 & 1 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\boldsymbol{W}^{(2)} = \begin{pmatrix} 3 & -2 \end{pmatrix}, \quad \boldsymbol{b}^{(2)} = \begin{pmatrix} -1 \end{pmatrix}$$

Entrada: $\boldsymbol{x} = (1, -1)^\top$

Passo 1 — pré-ativação camada 1:

$$\boldsymbol{z}^{(1)} = \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)} = \begin{pmatrix} 1(1) + 2(-1) \\ -1(1) + 1(-1) \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$$

Passo 2 — ativação ReLU:

$$\boldsymbol{h}^{(1)} = \max(\boldsymbol{z}^{(1)}, 0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Passo 3 — pré-ativação saída:

$$z^{(2)} = \boldsymbol{W}^{(2)}\boldsymbol{h}^{(1)} + b^{(2)} = 3(0) + (-2)(0) + (-1) =$$

Passo 4 — ativação sigmoide:

$$\hat{y} = \sigma(-1) = \frac{1}{1 + e^1} \approx \boxed{0.269}$$

Interpretação

Por que Funções de Ativação São Necessárias?

Teorema: Composição de Funções Lineares é Linear

Se $f^{(\ell)}(\boldsymbol{z}) = \boldsymbol{z}$ (identidade), então:

$$f(\boldsymbol{x}) = \boldsymbol{W}^{(L)}(\boldsymbol{W}^{(L-1)}(\cdots \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)} \cdots)) = \underbrace{\boldsymbol{W}_{\text{eq}}}_{\text{único } W} \boldsymbol{x} + \boldsymbol{b}_{\text{eq}}$$

Uma rede profunda linear é equivalente a uma rede de camada única! A profundidade não acrescenta poder expressivo.

Requisitos de uma Boa Ativação

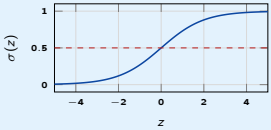
- **Não-linearidade**: poder de aproximação universal
- **Diferenciabilidade**: viabiliza backprop
- **Gradientes estáveis**: evita vanishing/exploding
- **Eficiência computacional**

Teorema de Aproximação Universal

Um MLP com **uma camada oculta** e ativação não-linear pode aproximar qualquer função contínua em um compacto de $\mathbb{R}^n$ com precisão arbitrária (Hornik, 1991).
Mas: pode precisar de exponencialmente muitos neurônios.

Funções de Ativação Clássicas

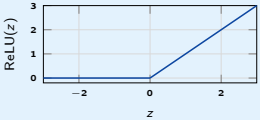
Sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$



$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$

Problema: saturação $\Rightarrow$ *vanishing gradient*

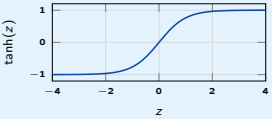
ReLU $f(z) = \max(0, z)$ (Nair & Hinton, 2010)



$f'(z) = 1[z > 0]$ — gradiente constante para $z > 0$!

Problema: neurônios mortos (*dying ReLU*)

Tangente Hiperbólica $\tanh(z)$



$\tanh(z) = 2\sigma(2z) - 1$, centrada em zero.

Melhor que sigmoid, mas ainda satura.

Variantes Modernas

Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$
ELU	z se $z > 0$; $\alpha(e^z - 1)$ caso contrário
GELU	$z\,\Phi(z)$ (usado em Transformers)
Swish	$z\,\sigma(\beta z)$ (Google, 2017)

Softmax — Ativação para Classificação Multiclasse

Definição

$$\text{softmax}(\boldsymbol{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K$$

Propriedades

- Saída $\in (0,1)$ e $\sum_k \text{softmax}(\boldsymbol{z})_k = 1$
- Interpreta-se como **distribuição de probabilidade**
- Softmax é monotônica: preserva ordenação de $\boldsymbol{z}$
- Combinada com **cross-entropy**: gradientes limpos

Exemplo Numérico

$\boldsymbol{z} = (2.0,\ 1.0,\ 0.1)$

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.1} = 1.105$$

$$S = 11.212$$

$$\text{softmax} = (0.659,\ 0.242,\ 0.099)$$

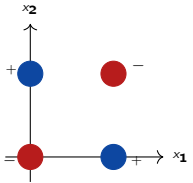
Classe 1 tem probabilidade $\approx 65.9\%$.

Estabilidade Numérica

XOR: Por que o Perceptron Simples Falha

Tabela-Verdade XOR

x_1	x_2	XOR	Classe
0	0	0	$-$
0	1	1	$+$
1	0	1	$+$
1	1	0	$-$



Não linearmente separável!

Prova Formal de Impossibilidade

Um perceptron linear busca w_1, w_2, b tal que:

$$
\begin{aligned}
w_1(0) + w_2(0) + b \leq 0 \quad & (0,0) \to - \\
w_1(0) + w_2(1) + b > 0 \quad & (0,1) \to + \\
w_1(1) + w_2(0) + b > 0 \quad & (1,0) \to + \\
w_1(1) + w_2(1) + b \leq 0 \quad & (1,1) \to -
\end{aligned}
$$

Somando linhas 1 e 4: $w_1 + w_2 + 2b \leq 0$
Somando linhas 2 e 3: $w_1 + w_2 + 2b > 0$
Contradição $\Rightarrow$ impossível!

Conclusão (Minsky & Papert, 1969)

Perceptrons de camada única são fundamentalmente limitados para problemas não-linearmente separáveis. $\Rightarrow$ Precisamos de **camadas ocultas**.

XOR Resolvido por MLP — Via Equações Normais

Arquitetura: $2 \to 2 \to 1$ com ReLU oculta, sigmoide na saída

Solução clássica (Goodfellow et al., Eq. 6.1):

$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \ \boldsymbol{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

$$\boldsymbol{W}^{(2)} = \begin{pmatrix} 1 & -2 \end{pmatrix}, \ \boldsymbol{b}^{(2)} = 0$$

Para $\boldsymbol{x} = (0, 1)$:

$$\boldsymbol{z}^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \ \boldsymbol{h}^{(1)} = \mathrm{ReLU}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$z^{(2)} = 1(1) + (-2)(0) + 0 = 1 \Rightarrow \hat{y} = 1 \ (\checkmark)$$

Tabela completa:

x_1	x_2	$h_1^{(1)}$	$h_2^{(1)}$	$z^{(2)}$	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	2	1	0	0

Insight Fundamental

A camada oculta **recodifica o espaço de entrada** criando uma representação linearmente separável. Esta é a essência do aprendizado de representações!

Princípios de Design de Arquiteturas

Hiperparâmetros Estruturais

- **Profundidade** L: número de camadas
- **Largura** d_ℓ: neurônios por camada
- **Tipo de ativação** por camada
- **Conectividade**: densa, esparsa, convolucional
- **Skip connections** (ResNets)

Profundidade vs. Largura

- Redes mais **profundas** $\Rightarrow$ hierarquia de features
- Redes mais **largas** $\Rightarrow$ memorização bruta
- Empiricamente: profundidade $>$ largura para generalização
- Teoricamente: funções exponencialmente mais expressivas com camadas (Montufar et al.,

Regras Práticas

Saída regressão	1 neurônio linear
Classificação bin.	1 neurônio + sigmoid
Classif. multi	K neurônios + softmax
Largura oculta	$64 \to 256 \to 1024$ (decrescente)
Profundidade	2–5 para tabular; mais para imagem

Aviso: Não há Teoria Completa

O design de arquitetura é amplamente empírico. Técnicas como NAS (*Neural Architecture Search*) automatizam parte da busca.

Teoria das Funções de Perda — Motivação

Princípio de Máxima Verossimilhança (MLE)

Dado um dataset $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, buscamos:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \log p(\mathcal{D}; \boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{N} \log p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

$$\equiv \arg\min_{\boldsymbol{\theta}} -\frac{1}{N} \sum_{i=1}^{N} \log p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta})$$

Conexão com Entropia Cruzada

Maximizar log-verossimilhança é equivalente a minimizar a **divergência KL** entre a distribuição empírica dos dados e a distribuição do modelo:

$$D_{KL}(p_{\text{dados}} \,\|\, p_{\text{modelo}}) = -\mathbb{E}_{p_{\textbf{dados}}}[\log p_{\text{modelo}}] + \text{const}$$

A função de perda emerge naturalmente da escolha do modelo probabilístico!

Perdas para Regressão

Mean Squared Error (MSE) — ℓ_2

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

- Assume $p(y|x) = \mathcal{N}(\hat{y}, \sigma^2)$ (ruído Gaussiano)
- Penaliza erros grandes **quadraticamente**
- **Sensível a outliers** (quadrado amplifica)
- Gradiente suave e convexo
- Mais usado na prática para regressão

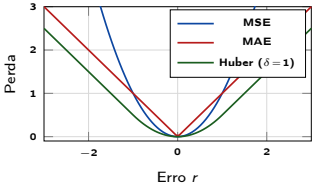
Mean Absolute Error (MAE) — ℓ_1

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N}\sum_{i=1}^{N}\left|y^{(i)} - \hat{y}^{(i)}\right|$$

Huber Loss (Combinação)

$$\mathcal{L}_{\delta} = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta\left(|r| - \frac{\delta}{2}\right) & |r| > \delta \end{cases}$$

$r = y - \hat{y}$. Quadrática perto de zero, linear para outliers.



Perdas para Classificação

Binary Cross-Entropy (BCE)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_i \left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

- Para classificação binária com saída sigmoid
- Deriva de $p(y|\boldsymbol{x}) = \text{Bernoulli}(\hat{y})$
- Gradiente: $\hat{y} - y$ (elegante com sigmoid!)

Categorical Cross-Entropy (CCE)

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_i \sum_{k=1}^{K} y_{i,k} \log \hat{y}_{i,k}$$

- Para K classes com saída softmax
- Com *one-hot*: $\mathcal{L} = -\frac{1}{N}\sum_i \log \hat{y}_{i,c_i}$

Regra Geral

Regressão	MSE ou Huber
Classif. binária	BCE + sigmoid
Classif. multi	CCE + softmax

Histórico do Backpropagation

Linha do Tempo

- **1960**: Kelley e Bryson — princípio da cadeia em controle ótimo
- **1970**: Linnainmaa — diferenciação automática reversa
- **1974**: Werbos — aplica em redes neurais (dissertação)
- **1986**: **Rumelhart, Hinton & Williams** — populariza em redes neurais
- **1988**: LeCun aplica em reconhecimento de dígitos
- **2000s**: Frameworks de diferenciação automática (Theano)
- **2015**: TensorFlow, PyTorch — autograd automático

O que é Backprop?

Não é um algoritmo de otimização — é um método eficiente para calcular gradientes usando a **regra da cadeia** de forma reversa ("backward pass").

Complexidade

- *Forward pass*: $O(|\boldsymbol{\theta}|)$
- *Backward pass*: $O(|\boldsymbol{\theta}|)$
- Custo total: constante $\times$ forward!

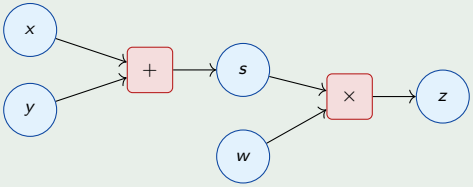
Grafos Computacionais

Definição

Um **grafo computacional** é um DAG (Grafo Acíclico Dirigido) onde:

- **Nós**: variáveis (escalares, vetores, tensores)
- **Arestas**: operações computacionais
- A topologia determina a ordem de avaliação e dos gradientes

Exemplo: $z = (x + y) \cdot w$



Diferenciação em Grafos

Para calcular $\frac{\partial \mathcal{L}}{\partial x}$, percorremos o grafo de trás para frente:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial x} = \frac{\partial \mathcal{L}}{\partial z} \cdot w \cdot 1$$

PyTorch vs TensorFlow

PyTorch: grafo *dinâmico* (define-by-run)
TensorFlow 1.x: grafo *estático* (define-then-run)
TF 2.x / JAX: ambos modos disponíveis

`tensor.backward()` em PyTorch percorre automaticamente o grafo e acumula gradientes em `.grad`.

Backpropagation — Matemática Completa

Regra da Cadeia Vetorial (Jacobiana)

Para $\boldsymbol{z} = f(\boldsymbol{x})$ e $\mathcal{L}$ escalar:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \left(\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}}\right)^{\top} \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$$

onde $\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} \in \mathbb{R}^{m \times n}$ é a matriz Jacobiana.

Gradientes por camada:
Para a camada ℓ: $\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)} \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}$,
$\boldsymbol{h}^{(\ell)} = f(\boldsymbol{z}^{(\ell)})$

Definimos $\boldsymbol{\delta}^{(\ell)} \equiv \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(\ell)}}$ (*sinal de erro*)

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(L)}} \quad \text{(depende da perda)}$$

$$\boldsymbol{\delta}^{(\ell)} = \left(\boldsymbol{W}^{(\ell+1)}\right)^{\top} \boldsymbol{\delta}^{(\ell+1)} \odot f'\left(\boldsymbol{z}^{(\ell)}\right)$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\boldsymbol{h}^{(\ell-1)}\right)^{\top}$$

Interpretação de $\odot$

$\odot$ denota produto elemento a elemento (Hadamard). A multiplicação por $f'(\boldsymbol{z}^{(\ell)})$ é a **gate** do gradiente.

Vanishing Gradient

Se $\max |f'| < 1$ (sigmoid: ≤ 0.25), então $\|\boldsymbol{\delta}^{(\ell)}\|$ decresce exponencialmente com a profundidade $L - \ell$. $\Rightarrow$ **ReLU** resolve em grande parte este

Backprop — Exemplo Numérico Completo

Rede: $x \to h = \text{ReLU}(wx + b) \to \hat{y} = h \cdot v \to \mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$
Valores: $x = 2,\ w = 1,\ b = -1,\ v = 3,\ y = 5.$

Forward pass:

$$z = wx + b = 1(2) + (-1) = 1$$
$$h = \text{ReLU}(1) = 1$$
$$\hat{y} = vh = 3(1) = 3$$
$$\mathcal{L} = \tfrac{1}{2}(3 - 5)^2 = 2$$

Backward pass:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 3 - 5 = -2$$
$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot h = (-2)(1) = -2$$
$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot v = (-2)(3) = -6$$
$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial h} \cdot 1[z > 0] = (-6)(1) = -6$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x = (-6)(2) = -12$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z} \cdot 1 = -6$$

Atualização (SGD com $\eta = 0.1$)

$w \leftarrow 1 - 0.1(-12) = 2.2, \quad b \leftarrow -1 - 0.1(-6) = -0.4, \quad v \leftarrow 3 - 0.1(-2) = 3.2$

Penalidades de Norma de Parâmetros

Regularização ℓ_2 (Weight Decay)

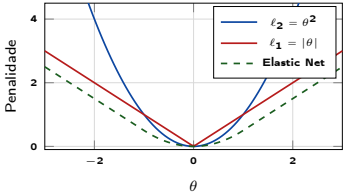
$$\tilde{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$$

- Assume prior $\boldsymbol{\theta} \sim \mathcal{N}(0, \lambda^{-1}\mathrm{I})$ (MAP)
- Penaliza pesos grandes igualmente
- Gradiente extra: $\lambda\boldsymbol{\theta}$
- Efeito: encolhe pesos em direção a zero (*shrinkage*)
- SGD com ℓ_2: "*weight decay*"

Regularização ℓ_1 (Lasso)

$$\tilde{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1$$

- Assume prior Laplace
- Promove **esparsidade** (muitos pesos = 0)
- Menos usado em deep learning



Elastic Net

Combinação: $\lambda_1 \|\boldsymbol{\theta}\|_1 + \lambda_2 \|\boldsymbol{\theta}\|_2^2$. Tem benefícios de ambas as penalidades.

Dropout

Definição (Srivastava et al., 2014)

Durante o treino, cada neurônio (exceto saída) é zerado com probabilidade p (tipicamente 0.5):

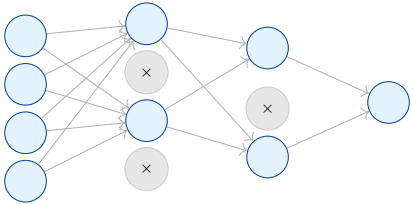
$$\tilde{h}_j = \frac{r_j \cdot h_j}{1-p}, \quad r_j \sim \text{Bernoulli}(1-p)$$

O fator $1/(1-p)$ (*inverted dropout*) garante que $\mathbb{E}[\tilde{h}_j] = h_j$.

Na Inferência

Dropout é desligado. Use todos os neurônios com pesos completos. O ensemble implícito é "integrado" automaticamente.

Treino (dropout)



Interpretação

Dropout treina implicitamente um **ensemble** exponencial de 2^n sub-redes com pesos compartilhados.

Data Augmentation e Early Stopping

Data Augmentation

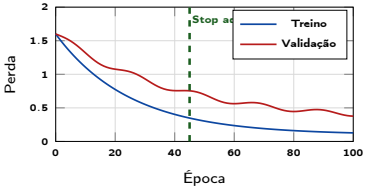
Ideia: Gerar novos exemplos de treino via transformações que preservam o rótulo:

- Imagens: flip, rotação, crop, jitter, mixup, cutout
- Texto: paráfrase, back-translation, sinônimos
- Tabular: SMOTE, Gaussian noise
- Áudio: pitch shift, time stretch

Efeito: aumenta efetivamente o tamanho do dataset e injeta invariâncias desejadas.

Early Stopping

Monitora perda de validação e interrompe treino quando há piora:



Early stopping é regularização implícita: equivale aproximadamente a ℓ_2 com $\lambda \propto 1/t$.

PyTorch — Conceitos Fundamentais

Tensores e Autograd

```python
import torch
import torch.nn as nn

# Tensor com gradiente
x = torch.tensor([1.0, 2.0],
                 requires_grad=True)
W = torch.randn(3, 2,
                requires_grad=True)
b = torch.zeros(3,
                requires_grad=True)

# Forward
z = W @ x + b
y = torch.relu(z)

# Backward (calcula gradientes)
loss = y.sum()
loss.backward()

# Gradientes disponiveis
print(W.grad)  # dL/dW
print(x.grad)  # dL/dx
```

Componentes Principais

`torch.Tensor`	Tensor N-dimensional com autograd
`nn.Module`	Classe base para camadas/modelos
`nn.Linear`	Camada totalmente conectada
`nn.ReLU/Sigmoid`	Ativações
`optim.SGD/Adam`	Otimizadores
`DataLoader`	Iterador sobre datasets
`loss.backward()`	Computa gradientes
`optim.step()`	Atualiza pesos

Dica: `.zero_grad()`

PyTorch **acumula** gradientes. Sempre chame `optimizer.zero_grad()` antes de `backward()`!

PyTorch — Loop de Treinamento Padrão

```python
import torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import DataLoader

# 1. Definir modelo
model = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 128), nn.ReLU(),
    nn.Linear(128, 10)
)

# 2. Perda e otimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# 3. Loop de treino
def train_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0.0
    for X_batch, y_batch in loader:
        optimizer.zero_grad()           # Zera gradientes acumulados
        logits = model(X_batch)         # Forward pass
        loss = criterion(logits, y_batch)  # Calcula perda
        loss.backward()                 # Backpropagation
        optimizer.step()                # Atualiza pesos
        total_loss += loss.item()
    return total_loss / len(loader)
```

MNIST — Carregamento e Preparação dos Dados

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Transformacoes: tensor + normalizacao (media=0.1307, std=0.3081)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Download automatico do MNIST
train_dataset = datasets.MNIST(root='./data', train=True,
                               download=True, transform=transform)
test_dataset  = datasets.MNIST(root='./data', train=False,
                               download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64,
                          shuffle=True,  num_workers=2)
test_loader  = DataLoader(test_dataset,  batch_size=1000,
                          shuffle=False, num_workers=2)

print(f"Treino: {len(train_dataset)} amostras")
print(f"Teste:  {len(test_dataset)} amostras")
print(f"Formato de uma imagem: {train_dataset[0][0].shape}")
```

MNIST — Arquitetura do MLP

```python
class MLP_MNIST(nn.Module):
    """MLP para classificacao do MNIST.

    Arquitetura: 784 -> 512 -> 256 -> 128 -> 10
    Ativacao: ReLU nas camadas ocultas
    Regularizacao: Dropout(0.3)
    """
    def __init__(self, dropout_p: float = 0.3):
        super().__init__()
        self.flatten = nn.Flatten()          # [B,1,28,28] -> [B,784]
        self.network = nn.Sequential(
            nn.Linear(784, 512),
            nn.ReLU(),
            nn.Dropout(p=dropout_p),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(p=dropout_p),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 10)               # logits (sem softmax)
        )
        self._init_weights()

    def _init_weights(self):
        """Inicializacao de Kaiming para camadas ReLU."""
        for m in self.modules():
```

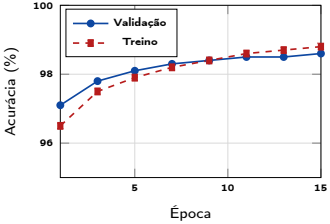
MNIST — Treinamento Completo

```python
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            total_loss += criterion(logits, y).item()
            correct += (logits.argmax(1) == y).sum().item()
    return total_loss/len(loader), correct/len(loader.dataset)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = MLP_MNIST(dropout_p=0.3).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

EPOCHS = 15
for epoch in range(1, EPOCHS + 1):
    model.train()
    train_loss = 0.0
    for X, y in train_loader:
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        loss = criterion(model(X), y)
        loss.backward()
```

MNIST — Resultados e Visualização

Resultados Esperados

Configuração	Acurácia Teste	Épocas
MLP 784-512-256-128-10	∼98.2%	15
+ Dropout(0.3)	∼98.4%	15
+ Weight Decay	∼98.5%	15
+ LR Scheduler	∼98.6%	15




```python
# Visualizar predicoes
import matplotlib.pyplot as plt

model.eval()
X_sample, y_sample = next(iter(test_loader))
with torch.no_grad():
    preds = model(X_sample.to(device)
            ).argmax(1).cpu()

fig, axes = plt.subplots(2, 5, figsize=(10,4))
for ax, img, pred, true in zip(
        axes.flat, X_sample[:10],
```

Referência MNIST

Estado da arte: $> 99.7\%$ (CNN). MLPs atingem ∼98.5% — suficiente para entender os fundamentos sem convolução.

Resumo da Aula

Parte I — Teórico

- ✓ Contexto histórico: McCulloch-Pitts → Deep Learning
- ✓ Formalismo: camadas, pesos, ativações
- ✓ Cálculos manuais: forward pass
- ✓ Funções de ativação: sigmoid, ReLU, softmax
- ✓ XOR: prova de limitação + solução MLP
- ✓ Design de arquiteturas
- ✓ Loss functions: MSE, BCE, CCE, Huber, Focal
- ✓ Backpropagation: história e matemática
- ✓ Grafos computacionais
- ✓ Regularização: ℓ_1, ℓ_2, Dropout, Early Stopping, Augmentation

Parte II — Prática

- ✓ PyTorch: tensores e autograd
- ✓ `nn.Module`, `DataLoader`, loop de treino
- ✓ Implementação MLP para MNIST (∼98.5%)

Próximas Aulas

- Redes Convolucionais (CNN)
- Otimização avançada (Adam, RMSProp, momentum)
- Batch Normalization
- Arquiteturas modernas (ResNet, Transformers)

Leitura Obrigatória

Goodfellow et al., Cap. 6 (*Deep Feedforward*

Referências Bibliográficas

- I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016. `deeplearningbook.org`
- D. Rumelhart, G. Hinton, R. Williams. "Learning representations by back-propagating errors." *Nature*, 323, 1986.
- M. Minsky, S. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.
- K. Hornik. "Approximation capabilities of multilayer feedforward networks." *Neural Networks*, 4(2), 1991.
- N. Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *JMLR*, 15(1), 2014.
- V. Nair, G. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines." *ICML*, 2010.

Obrigado!

Dúvidas?

Baseado em: Goodfellow, Bengio & Courville — *Deep Learning* (MIT Press, 2016)
Código disponível em: `github.com/.../mlp-lecture`